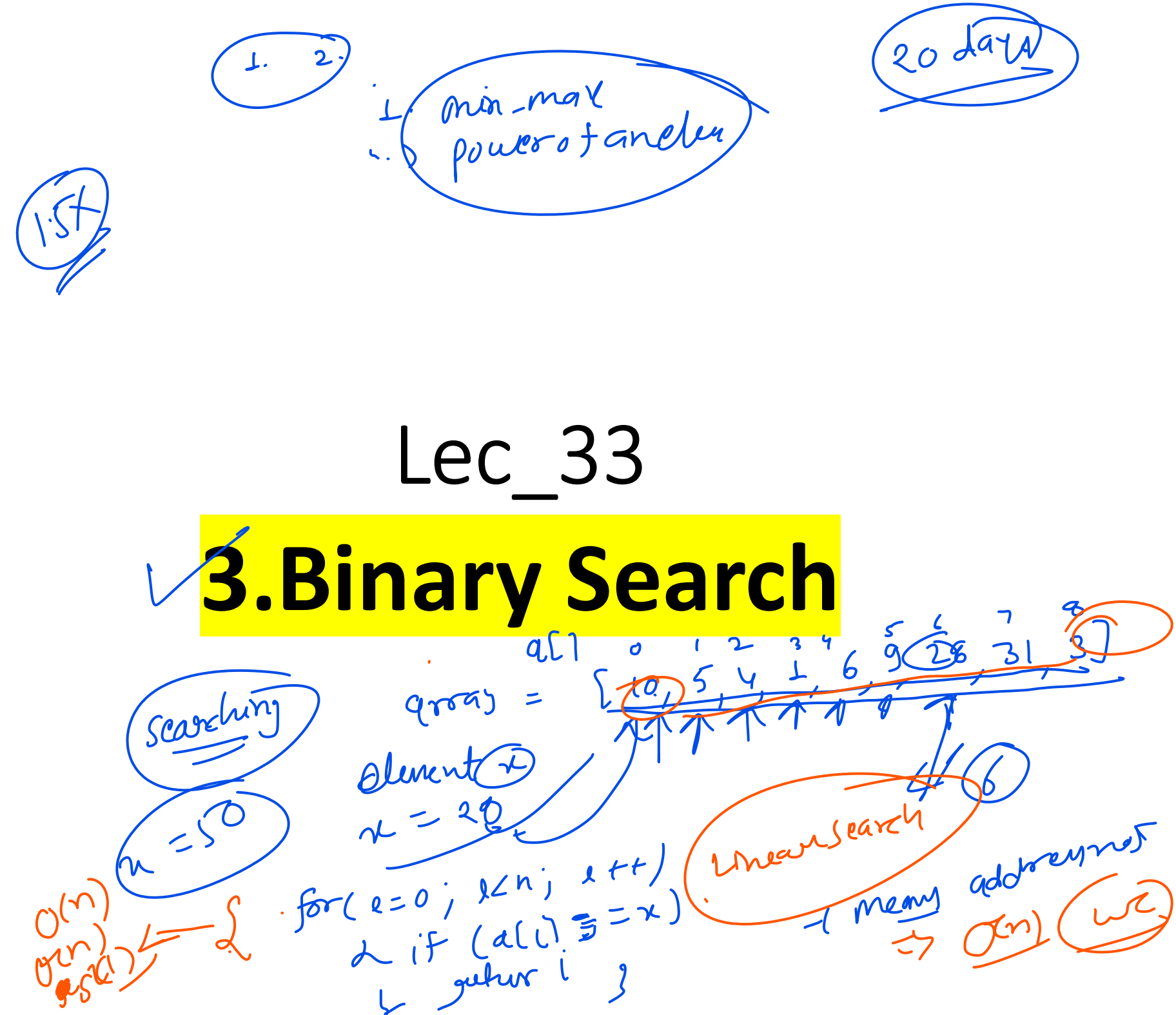

1. 2.
1.
min_max
power of ancles
20 days
1st
Lec_33
3.Binary Search
searching
n = 50
O(n)
O(n)
O(1)
array =
element (x)
x = 20
for(i=0; i<n; i++)
if (a[i] == x)
return i
}
LinearSearch
many addressings
O(n)
WC

return -1

(LS) Linear search TC = O(n)
(BS) Binary search TC = O(log n)

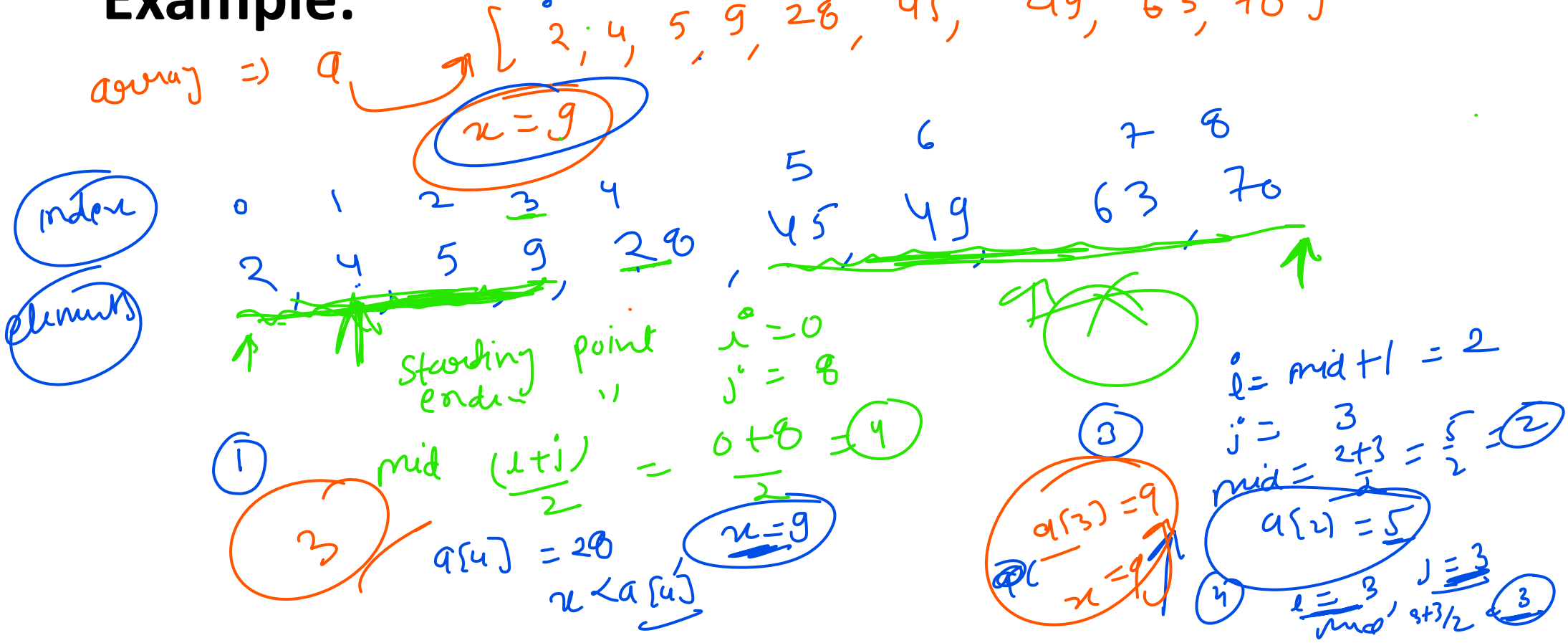
log n << n

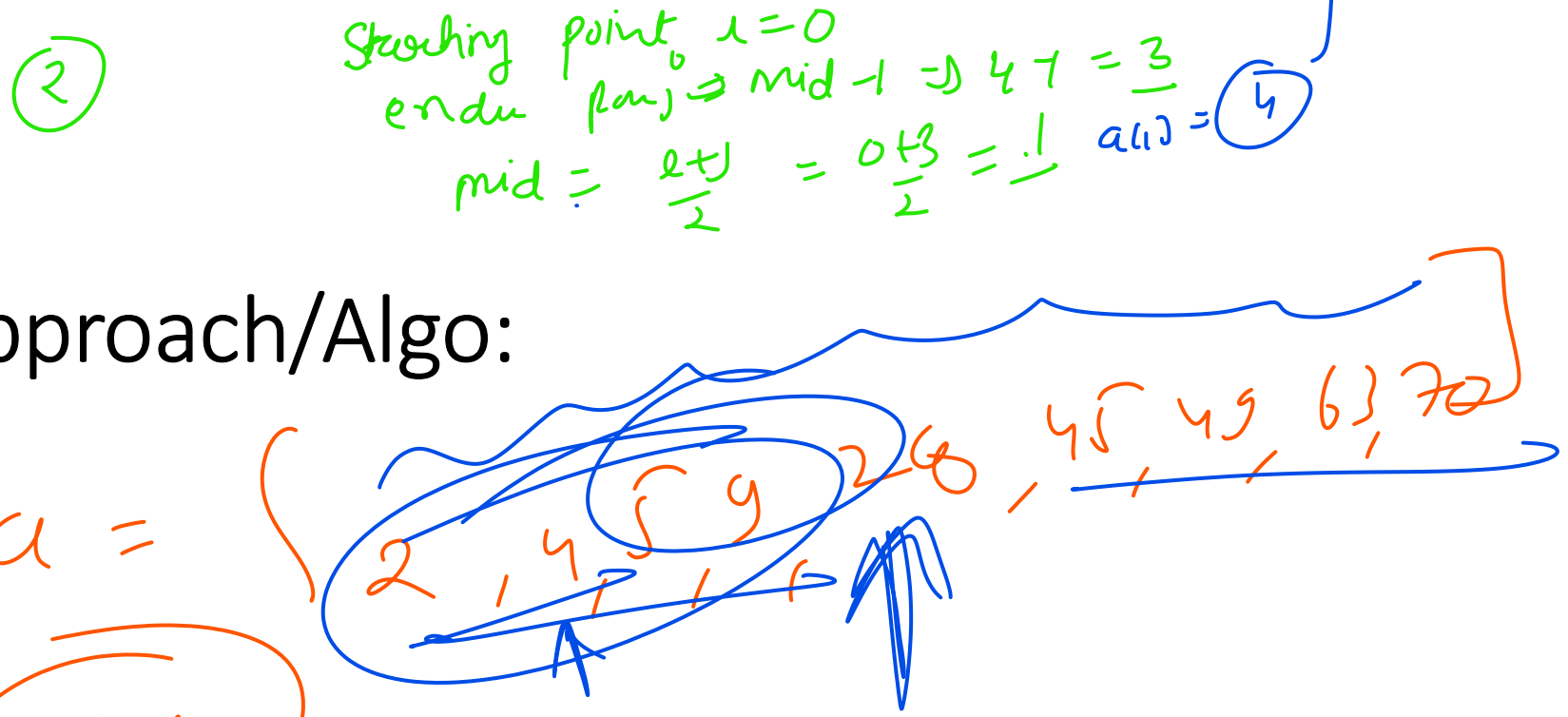
Input: Sorted Array of n elements , an element x

(index)

Output: Location of Element x , if Exist, if not return -1

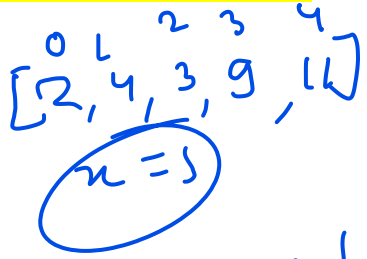
Example:

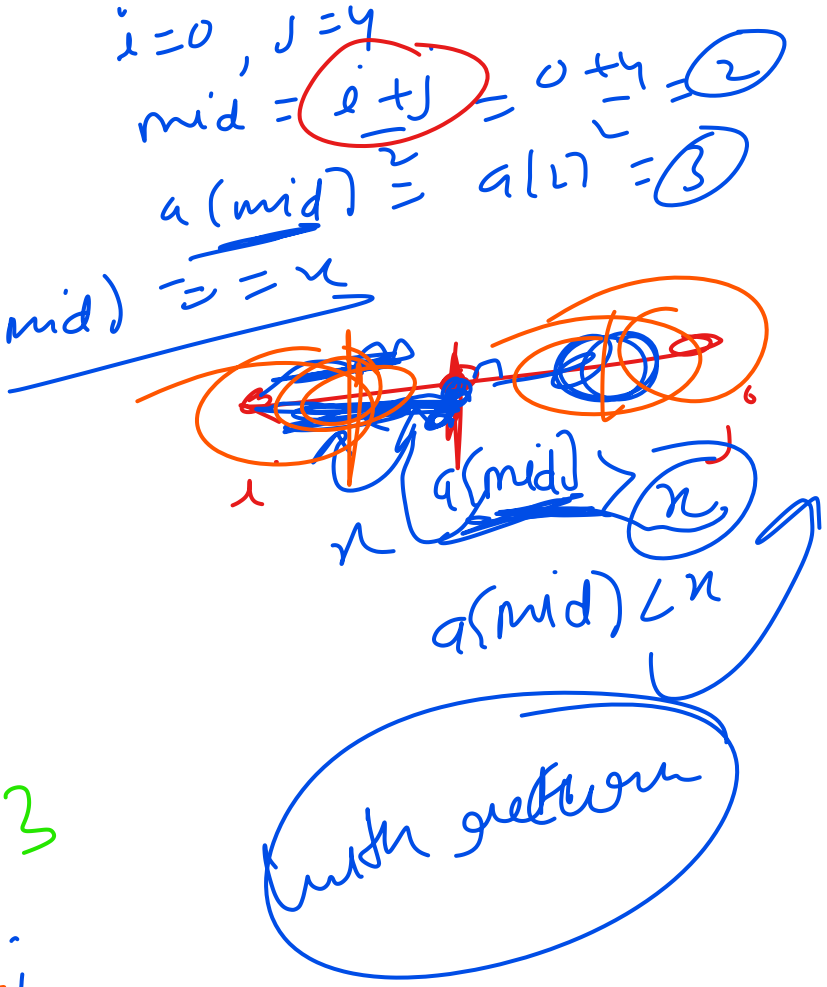




Approach/Algo:

Algorithm With Recursion:






```
BS(a, i, j, x)
{
    if (i == j)
    { if(a[i] == x) { return i }
      else return -1;
    }
    else
    { mid = (i+j)/2
      if (a[mid] == x) { return mid; }
      if (a[mid] > x)
        return BS(a, i, mid-1, x);
```

$T(c) = 1$

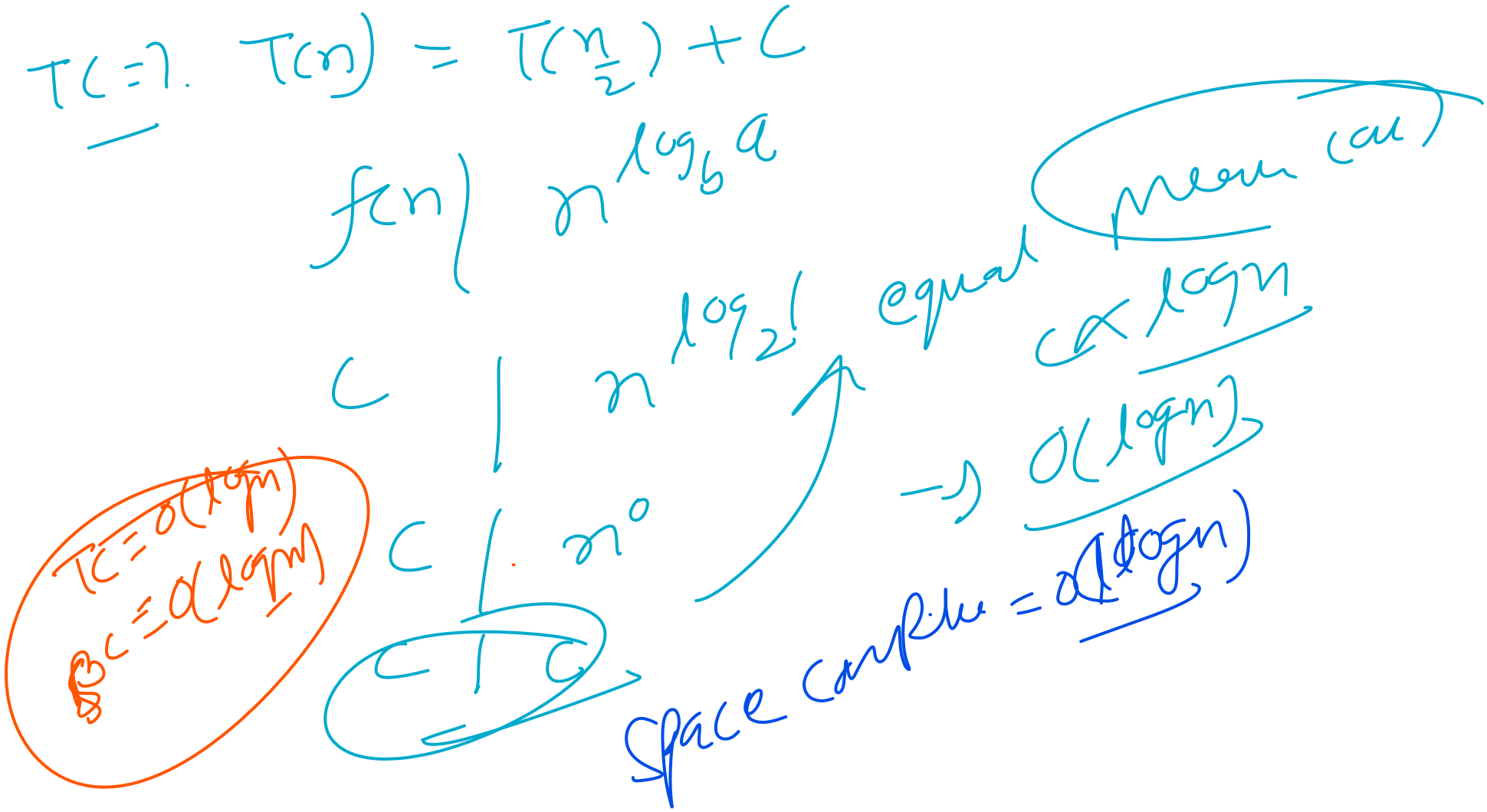
$T(n) = c + T(n/2)$

else if (a[mid] < x)
return BS(a, mid+1, j, x);

Time complexity

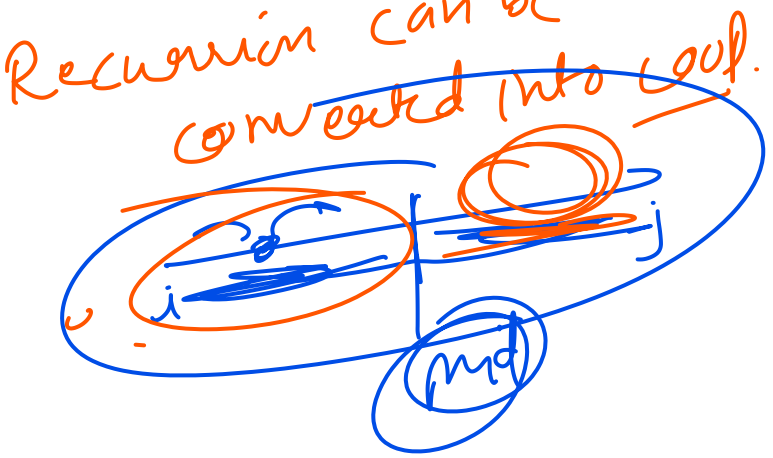
$$T(n) = T\left(\frac{n}{2}\right) + C$$

Time Space Complexity Analysis:



Algo Without Recursion:

```
BS(a,i,j,x)
{ while(i<=j)
  { if(i==j) { if(a[i]==x) return i;
               else return -1;
  }
  mid = (i+j)/2
  if (a[mid]==x) return mid;
  if (a[mid]>x)
      j = mid-1;
  else i = mid+1;
}
```



$TC = O(\log n)$

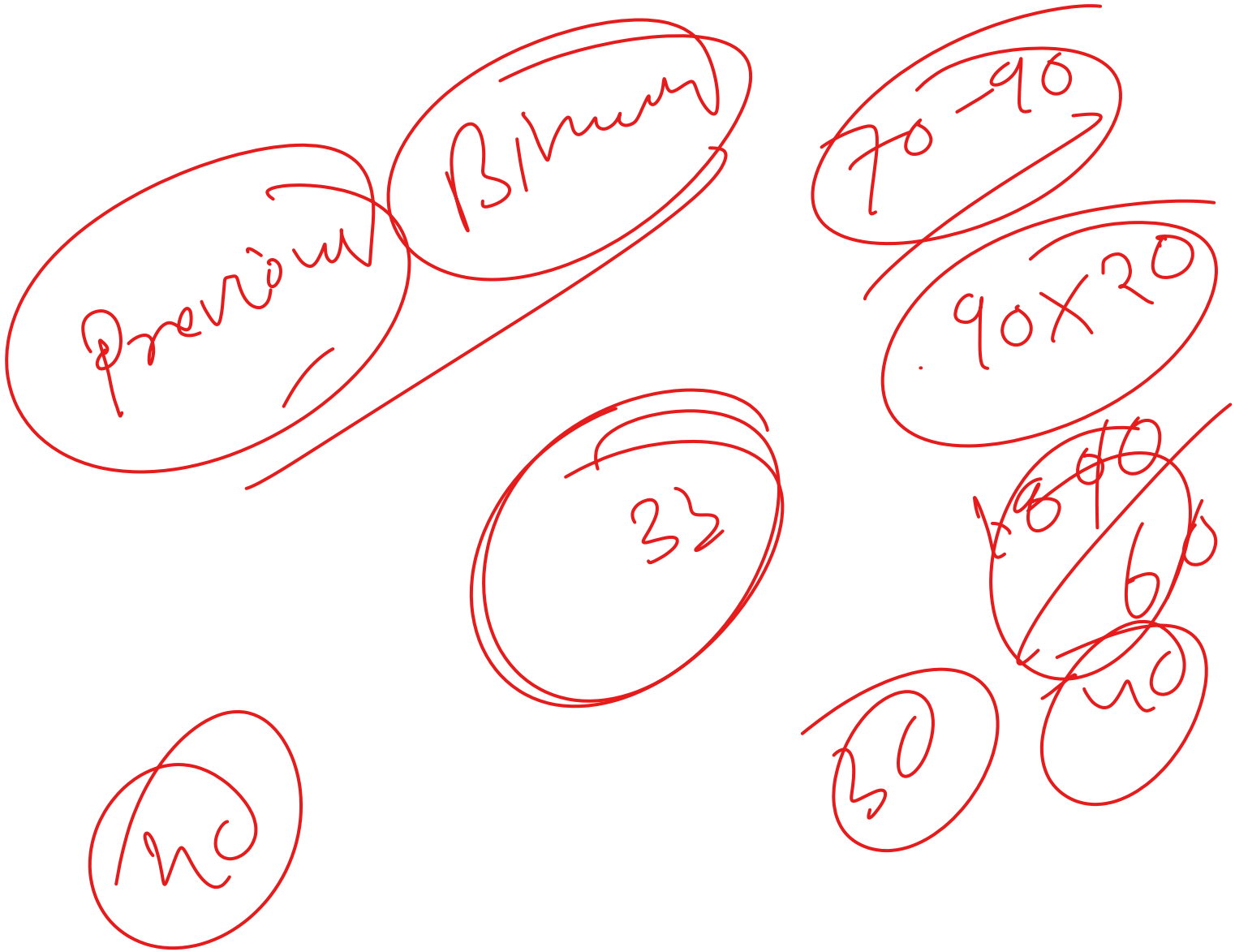
Linear vs Binary: TC and SC Comparisons:

Linear	Binary search
TC, BC => O(1)	O(1)
TC, WC => O(n)	O(logn)

Space complexity:..

Linear = O(1)

Binary = O(logn) → Stack needed for the recursion


Previous
Bikum
70-90
90x20
33
60
20
20